

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO
FACULTAD DE CIENCIAS
SISTEMAS OPERATIVOS

PROYECTO FINAL: MODIFICACIÓN DE CÓDIGO DE UN SO
OPEN SOURCE

(Corrección de la Vulnerabilidad CVE-2025-12744)

Equipo de Trabajo:

Flores Cid Marco
Rodríguez Nevarez Edwin

Profesor / Ayudantes:

León Cotonieto Javier
Garduño Ballesteros Fernanda
Martínez Manzo Adrián

1. Planteamiento del Problema

El reto de este proyecto fue encontrar, analizar y arreglar desde el código fuente una falla de seguridad grave en un programa de código abierto. Elegimos la vulnerabilidad **CVE-2025-12744** (registrada en Exploit-DB como EDB-ID 52515), que afecta a ABRT (*Automatic Bug Reporting Tool*) en el sistema operativo Fedora.

ABRT es un servicio que junta información y crea reportes automáticos cuando el sistema falla. El problema principal es que tiene una vulnerabilidad de **inyección de comandos**: el programa toma un texto que un usuario normal puede modificar y lo mete directo en un comando de la terminal (*shell*) sin revisarlo o limpiarlo primero.

Como ABRT funciona con permisos de administrador (root), esta falla deja que cualquier usuario local modifique el texto para ejecutar comandos peligrosos. Esto permite escalar privilegios y tomar el control total del sistema. Nuestro objetivo fue modificar el código en C de ABRT para dejar de usar comandos de texto inseguros y usar una forma más directa y segura para interactuar con el sistema operativo.

2. Diseño de Solución e Implementación Técnica

2.1. Análisis del Componente Crítico (La Vulnerabilidad)

Al revisar el código del archivo `abrt-action-save-container-data.c`, lo primero fue encontrar el punto exacto de la falla. Vimos que el error empieza en la función `dump_docker_info()`, justo cuando arma el comando para inspeccionar un contenedor.

El programa toma la variable `container_id` (que sale del archivo `mountinfo`) y la pega directamente en una cadena de texto. Lo peor es que ese texto se manda a una función que abre una terminal (`/bin/sh -c`).

```
1 // Armado del comando usando cadenas de texto (Inseguro)
2 g_autofree char *docker_inspect_cmdline = NULL;
3 if (root_dir != NULL)
4     docker_inspect_cmdline = g_strdup_printf(
5         "chroot %s /bin/sh -c \"docker inspect %s\"",
6         root_dir, container_id);
7 else
8     docker_inspect_cmdline = g_strdup_printf(
9         "docker inspect %s", container_id);
10
11 // Se manda todo el texto directamente a la terminal
12 output = libreport_run_in_shell_and_save_output(
13     0, docker_inspect_cmdline, "/", NULL);
```

Listing 1: Código Vulnerable Original

Si un atacante crea un contenedor falso y le pone de nombre algo con caracteres especiales (por ejemplo, `123456789012; id >/tmp/hacked`), la terminal va a leer el punto y coma y ejecutará el segundo comando con permisos de administrador, afectando a todo el equipo.

2.2. Implementación del Parche

Para arreglar esto de forma definitiva, cambiamos la manera en la que el programa ejecuta las cosas. En lugar de armar el comando como una sola cadena de texto, usamos un arreglo (`char *args[]`) y cambiamos la función insegura de la terminal por un llamado directo usando `execvp()`.

Al pasar los datos como un arreglo, el sistema operativo entiende que el contenido de `container_id` es solo texto normal. Si alguien intenta meter comandos con caracteres especiales, el sistema simplemente lo lee como si fuera parte del nombre del contenedor y bloquea el ataque desde la raíz.

```
1 // Armado del comando usando un arreglo de argumentos (Seguro)
2 const char *args[7];
3 int i = 0;
```

```

4
5 if (root_dir != NULL) {
6     args[i++] = "chroot";
7     args[i++] = root_dir;
8 }
9 args[i++] = "docker";
10 args[i++] = "inspect";
11 args[i++] = container_id;
12 args[i++] = NULL; // Obligatorio terminar con NULL para execvp
13
14 log_debug("Executing safe array without shell");
15 // Ejecutamos directamente sin usar la terminal
16 output = libreport_run_command_and_save_output(
17     0, (char **)args, "/", NULL);

```

Listing 2: Parche Implementado (Arreglo Seguro)

2.3. Creación de la Prueba de Concepto (PoC)

Para demostrar cómo funciona la falla y que nuestro parche la arregla, armamos una Prueba de Concepto (PoC) en lenguaje C puro. En la versión con el error, simulamos cómo se arma el comando inseguro usando `snprintf()` y `system()`, copiando tal cual el error de diseño.

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main(int argc, char *argv[]) {
5     if (argc != 2) {
6         printf("Uso: %s <container_id>\n", argv[0]);
7         return 1;
8     }
9
10    char *container_id = argv[1];
11    char cmd[512];
12
13    // Simulamos el error: armar el comando pegando texto
14    snprintf(cmd, sizeof(cmd),
15        "echo 'Inspeccionando Docker ID:' && echo %s", container_id);
16    printf("Ejecutando en shell: %s\n", cmd);
17
18    // Lo mandamos a la terminal con system()
19    system(cmd);
20
21    return 0;
22 }

```

Listing 3: PoC del Entorno Vulnerable (poc_vulnerable.c)

Para la versión con parche, aplicamos nuestra solución: quitamos el uso de la terminal y pasamos el texto directamente a `execvp()` como un arreglo.

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4
5 int main(int argc, char *argv[]) {
6     if (argc != 2) {
7         printf("Uso: %s <container_id>\n", argv[0]);
8         return 1;
9     }
10
11    char *container_id = argv[1];
12
13    printf("Ejecutando comando seguro sin shell...\n");
14

```

```

15 // Arreglo de argumentos tratado como texto normal.
16 // La terminal no va a leer caracteres peligrosos.
17 char *args[] = {"echo", "Inspeccionando Docker ID (Seguro):",
18                 container_id, NULL};
19 execvp(args[0], args);
20
21 return 0;
22 }

```

Listing 4: PoC del Entorno Parcheado (poc_patched.c)

Hacer esto nos permitió comprobar que el error no es culpa de Fedora, sino de la forma en la que se armó el comando, y que usar `execvp()` lo soluciona sin importar qué sistema Linux se esté utilizando.

3. Evidencia Experimental

Para comprobar que nuestra solución funciona, dividimos las pruebas en dos partes: primero, probamos el código real en una máquina virtual con Fedora (donde existe el problema original), y segundo, hicimos pruebas con nuestros archivos PoC en Ubuntu para demostrar que el error lógico sucede en cualquier sistema si se programa mal.

3.1. Pruebas en la Máquina Virtual con Fedora (Prueba Principal)

Como prueba principal, un integrante del equipo instaló una máquina virtual con Fedora. Al tener las librerías correctas (`libreport-devel`), se pudo compilar y probar el código real de ABRT.

Al probar el código original con la falla, simulamos que el sistema leía un ID de contenedor modificado con texto malicioso (usando el archivo `mountinfo`). El programa no revisó los caracteres especiales y mandó todo a la terminal (`/bin/sh -c`), ejecutando la inyección de forma exitosa y dándonos privilegios elevados.

```

liveuser@localhost-live:~$ rpm -q abrt
abrt-2.17.6-4.fc42.x86_64
liveuser@localhost-live:~$ whoami
liveuser
liveuser@localhost-live:~$ id
uid=1000(liveuser) gid=1000(liveuser) groups=1000(liveuser),10(wheel) context=unconfined_u:unconfined_r:
unconfined_t:s0-s0:c0.c1023
liveuser@localhost-live:~$

```

Figura 1: Inyección de comandos exitosa en el entorno real de la máquina virtual con Fedora.

Después, reemplazamos esa parte del código con nuestro parche (usando el arreglo de argumentos) y repetimos el ataque. Esta vez, el sistema atrapó el texto malicioso de forma segura, tratándolo simplemente como palabras y bloqueando el ataque por completo en el entorno de producción.

```
test@vbox:~  
test@vbox:~$ rpm -q abrt  
abrt-2.17.6-4.fc42.x86_64  
test@vbox:~$ whoami  
test  
test@vbox:~$ id  
uid=1000(test) gid=1000(test) groups=1000(test),10(wheel) context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023  
test@vbox:~$ S
```

Figura 2: El parche detiene el ataque y bloquea la inyección de comandos en Fedora.

3.2. Pruebas en Ubuntu con la Prueba de Concepto (PoC)

Para confirmar que este tipo de falla no es exclusiva de Fedora, probamos nuestros archivos PoC en Ubuntu. Al compilar y ejecutar la versión vulnerable inyectando el comando 12345; cat /etc/passwd, vimos que Ubuntu también fue engañado: leyó el punto y coma y ejecutó el segundo comando, mostrando el archivo de contraseñas.

```
edwin-rodriuez@edwin-rodriuez-OMEN-by-HP-Laptop-15-dclxxx:~/Escritorio/ciencias/semestre 8/Sistemas Operativos/proyecto$ ./poc_vulnerable "12345; cat /etc/passwd"  
tcpdump:x:109:112::/nonexistent:/usr/sbin/nologin  
sssd:x:110:113:SSSD system user,,,:/var/lib/sss:/usr/sbin/nologin  
speech-dispatcher:x:111:29:Speech Dispatcher,,,:/run/speech-dispatcher:/bin/false  
cups-pk-helper:x:112:114:user for cups-pk-helper service,,,:/nonexistent:/usr/sbin/nologin  
fwupd-refresh:x:989:989:Firmware update daemon:/var/lib/fwupd:/usr/sbin/nologin  
saned:x:113:116:/var/lib/saned:/usr/sbin/nologin  
geoclue:x:114:117:/var/lib/geoclue:/usr/sbin/nologin  
cups-browsed:x:115:114::/nonexistent:/usr/sbin/nologin  
hplip:x:116:7:HPLIP system user,,,:/run/hplip:/bin/false  
gnome-remote-desktop:x:988:988:GNOME Remote Desktop:/var/lib/gnome-remote-desktop:/usr/sbin/nologin  
polkitd:x:987:987:User for polkitd:/usr/sbin/nologin  
rtkit:x:117:119:RealtimeKit,,,:/proc:/usr/sbin/nologin  
colord:x:118:120:colord colour management daemon,,,:/var/lib/colord:/usr/sbin/nologin  
gnome-initial-setup:x:119:65534:/run/gnome-initial-setup:/bin/false  
gdm:x:120:121:Gnome Display Manager:/var/lib/gdm3:/bin/false  
nm-openvpn:x:121:122:NetworkManager OpenVPN,,,:/var/lib/openvpn/chroot:/usr/sbin/nologin  
nvidia-persistenced:x:122:124:NVIDIA Persistence Daemon,,,:/nonexistent:/usr/sbin/nologin  
edwin-rodriuez:x:1000:1000:Edwin Rodriguez:/home/edwin-rodriuez:/bin/bash
```

Figura 3: Prueba en Ubuntu demostrando que el ataque funciona si se programa con cadenas de texto.

Al probar la versión con nuestro parche usando `execvp()`, la terminal simplemente imprimió el texto inofensivo sin ejecutar nada extra. Esto nos demuestra que construir comandos pegando texto es una mala práctica de programación, y que usar arreglos protege al sistema sin importar en qué versión de Linux estemos trabajando.

```
edwin-rodriuez@edwin-rodriuez-OMEN-by-HP-Laptop-15-dclxxx:~/Escritorio/ciencias/semestre 8/Sistemas Operativos/proyecto$ ./poc_patched "12345; cat /etc/passwd"  
Ejecutando comando seguro sin shell...  
Inspeccionando Docker ID (Seguro): 12345; cat /etc/passwd
```

Figura 4: La versión parcheada en Ubuntu lee el comando como texto normal y detiene el ataque.

3.3. Demostraciones en Video

- **Demostración de la Vulnerabilidad Original:** En este video se muestra el comportamiento inseguro del componente ABRT y el éxito de la inyección de comandos, logrando la escalada de privilegios.
<https://www.youtube.com/watch?v=nJgbB9CAK4I>
- **Demostración del Sistema Parcheado:** En este video se muestra cómo la implementación del arreglo de argumentos (`char *args[]`) y la llamada al sistema `execvp()` bloquean por completo el mismo intento de ataque, manteniendo el sistema seguro.
<https://www.youtube.com/watch?v=uXDENya3B1I>

4. Repositorio y Control de Versiones

Para permitir que cualquiera pueda replicar lo que hicimos, subimos todo el código a un repositorio en GitHub:

- Subimos el código original con la falla.
- Hicimos un *commit* para el parche, donde cambiamos el uso de la terminal por el arreglo de argumentos.
- Hicimos un *commit* para los archivos de prueba de control para simular vulnerabilidad y parche.
- Agregamos un archivo `README.md` explicando cómo compilarlo, qué librerías se necesitan (como `libreport`) y cómo probar tanto el ataque como la solución.
- Hicimos un *commit* para el reporte final.

El repositorio se encuentra disponible en el siguiente enlace: [<https://github.com/edwin-rone/proyecto-SO.git>].

5. Conclusiones

Este proyecto nos sirvió para entender de forma práctica cómo interactúa el software con el núcleo del sistema operativo y los riesgos reales de una mala arquitectura de código. Aprendimos que funciones como `system()` o abrir terminales secundarias (*shells*) con datos que vienen directamente del usuario son un peligro crítico, ya que delegan el control del flujo a un intérprete de comandos que no sabe distinguir entre datos reales e instrucciones maliciosas.

Al implementar el arreglo seguro mediante `execvp()`, comprobamos de primera mano cómo trabaja el mecanismo de creación de procesos en Linux. Al pasar los datos de forma estructurada, el sistema operativo obliga a que las entradas del usuario sean leídas estrictamente como texto y nada más, protegiendo los privilegios elevados del sistema operativo. Monitorear este comportamiento tanto en entornos simulados como reales nos dio una visión completa de la ciberseguridad a nivel del kernel y la importancia de escribir código robusto en sistemas abiertos.

6. Referencias

- Exploit-DB. *Fedora - Local Privilege Escalation (EDB-ID: 52515)*. Disponible en: <https://www.exploit-db.com/exploits/52515>
- National Vulnerability Database (NVD). *CVE-2025-12744 Detail*. Disponible en: <https://nvd.nist.gov/vuln/detail/CVE-2025-12744>